

EE4705 Human-Robot Interaction

Project Report

Group 6

Name	Matriculation Number
Chan Ping-Shuen Savannah	A0283223L
Singhal Yash	A0288249L
Muhammad Irfan Bin Abdullah	A0273164E

Task 1: Implementation of STOMP for Collision-Free Motion Planning of the Kinova Gen3 Manipulator - Savannah

1.1 Objective and Experimental Setup

The objective of Task 1 was to complete the provided MATLAB implementation of Stochastic Trajectory Optimisation for Motion Planning (STOMP) to generate a smooth, collision-free trajectory for the 7-DOF Kinova Gen3 robotic manipulator. Five incomplete functions were implemented: *helperSTOMP.m, updateJointsWorldPosition.m, stompSamples.m, stompObstacleCost.m, and stompDTheta.m*. The original collision-avoidance scenario and planning parameters were retained to evaluate the completed implementation.

The robot was required to move between two end-effector poses while avoiding a cubic obstacle positioned within its workspace. The initial and target end-effector positions were *(0.40,0,0.20)* m and *(0.35,0.55,0.35)* m, respectively, with both poses specifying a 180° rotation about the y-axis. A cube with side length 0.16 m was positioned at *(0.40,0.46,0.26)* m. The environment was represented using a voxel occupancy grid with a resolution of *0.02* m, from which a signed Euclidean distance transform (sEDT) was generated for obstacle-cost evaluation.

The initial joint-space trajectory was obtained through linear interpolation between the initial and target configurations calculated using inverse kinematics. STOMP was configured with 20 trajectory waypoints, 20 sampled trajectories per iteration, a convergence threshold of 0.1, and a maximum of 50 iterations.

1.2 STOMP Algorithm Implementation

The incomplete functions were implemented to establish the complete STOMP optimization pipeline, consisting of trajectory sampling, cost evaluation, probability weighting, and trajectory updates.

Forward kinematics *(updateJointsWorldPosition.m).* MATLAB's getTransform function was used to calculate the homogeneous transformation matrices of the robot's movable joints for each sampled configuration. The translation components were extracted to obtain joint positions in the robot base frame. These positions were subsequently used to construct the collision-sphere approximation of the manipulator.

Trajectory sampling *(stompSamples.m)*. Alternative trajectories were generated by introducing zero-mean multivariate Gaussian perturbations to the current trajectory, as seen in the equation below.

$$\tilde{\theta}_k = \theta + \epsilon_k, \qquad \epsilon_k \sim \mathcal{N}(0, \Sigma)$$

Figure 1.1: Trajectory sampling

The sampling covariance was derived from the inverse of the trajectory smoothness matrix, producing temporally correlated perturbations. Sampling was performed independently for each joint, while the initial and final configurations were preserved by setting the endpoint perturbations to zero.

Obstacle-cost evaluation *(stompObstacleCost.m)*. The manipulator links were approximated using overlapping spheres of radius 0.05 m. For each sphere, the sEDT was queried to determine its distance from the nearest obstacle. A piecewise obstacle penalty was applied based on the sphere's clearance, incorporating a 0.05 m safety margin. The penalty was weighted by the approximate workspace displacement of the sphere between consecutive waypoints, as seen in the equation below.

$$C_{\text{obs}}(t) = \sum_i \phi(d_i - r_i) \, \|\dot{\mathbf{x}}_i(t)\|$$

Figure 1.2: Obstacle-cost function

d_i denotes the signed distance at sphere (i), r_i denotes its radius, and ϕ denotes the piecewise clearance penalty. The implementation also assigned a large penalty to sampled sphere centres outside the voxel environment.

Probability-weighted trajectory updates *(stompDTheta.m).* The existing probability-update function assigned higher weights to sampled trajectories with lower local costs. These probabilities were combined with the corresponding perturbations to estimate the trajectory update.

$$\Delta\theta_{j,t} = \sum_{k=1}^{K} P_{k,t}\epsilon_{k,j,t}$$

Figure 1.3: Probability-weighted trajectory update

Where $P_{k,t}$ represents the probability of sample (k) at waypoint (t), and $\boldsymbol{\varepsilon}_{k,j,t}$ is its perturbation for joint (j). The resulting updates were subsequently smoothed using the inverse smoothness matrix.

Optimisation loop *(helperSTOMP.m)*. The completed function integrated the sampling, cost evaluation, probability calculation, and trajectory-update operations into an iterative optimisation process. The updated trajectory was evaluated after each iteration, with convergence determined by the change in total trajectory cost. Additional stopping conditions

were implemented for the maximum iteration count and negligible trajectory updates. The trajectory objective combined the state-dependent obstacle cost with a quadratic smoothness penalty.

$$Q(\boldsymbol{\theta}) = \sum_{t=1}^{N} S(\boldsymbol{\theta}_t) + \frac{1}{2} \operatorname{tr}\left(\boldsymbol{\Theta}_{\text{int}} R \boldsymbol{\Theta}_{\text{int}}^{T}\right)$$

Figure 1.4: Total trajectory objective

where θ_{int} is the matrix of interior joint-space waypoints, and $R = A^T A$ is the smoothness-cost matrix. This formulation encourages obstacle avoidance while penalising abrupt changes in joint motion.

1.3 Experimental Results

The completed STOMP planner successfully optimised the Kinova Gen3 trajectory under the original obstacle configuration. The algorithm converged after 11 iterations, reducing the total trajectory cost from 73.3944 to 52.8407, corresponding to a **28%** reduction. The total optimisation runtime was 887.40 seconds (~14.8 minutes).

Performance metric	Result
Robot	Kinova Gen3 (7-DOF)
Trajectory waypoints	20
Sampled trajectories per iteration	20
Optimization iterations	11
Initial trajectory cost	73.3944
Final trajectory cost	52.8407
Total cost reduction	28%
Final smoothness cost	52.4968
Colliding original waypoints	0 / 20
Colliding densely sampled configurations	0 / 191

Table 1.1: STOMP optimisation results

The trajectory cost decreased across successive iterations, with the largest reductions occurring during the earlier stages of optimisation. The smoothness cost remained approximately constant at 52.5, indicating that most of the improvement was associated with

reducing the state-dependent obstacle cost rather than substantially altering the smoothness penalty.

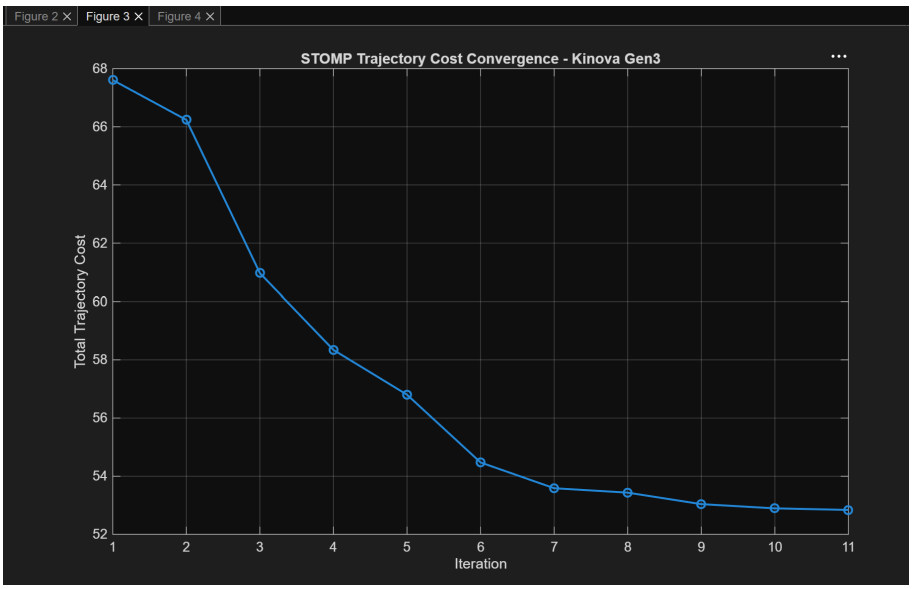


Figure 1.5: STOMP total trajectory cost against iteration

Collision verification was performed using MATLAB's checkCollision function. The optimised trajectory passed the original waypoint collision check, with no collisions detected across all 20 configurations. An additional verification was conducted by interpolating 10 samples per trajectory segment, resulting in 191 evaluated configurations. No collisions were detected in this denser evaluation.

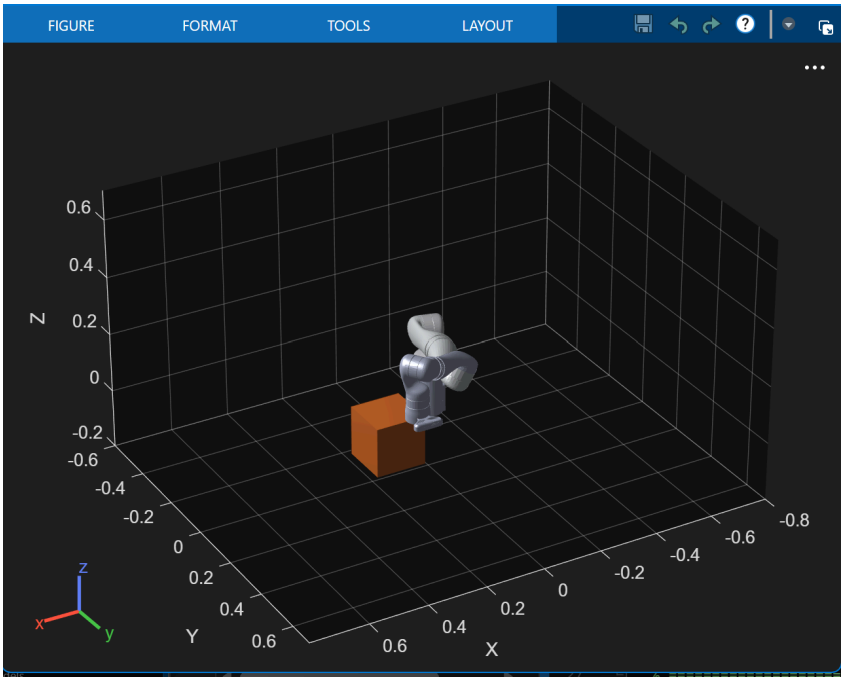


Figure 1.6: Kinova Gen3 manipulator and obstacle environment

The final trajectory was visualised using MATLAB's robot animation functions, and the optimised joint configurations were saved for subsequent playback and analysis.

1.4 Discussion and Limitations

The completed STOMP planner reduced trajectory cost by 28% and achieved zero detected environment collisions across 20 waypoints and 191 interpolated configurations, demonstrating effective collision avoidance in the original Kinova Gen3 scenario.

However, optimisation required 14.8 minutes, indicating substantial computational cost. Collision spheres only approximate the robot's geometry, while discrete collision checks cannot guarantee continuous collision-free motion. Self-collisions were also excluded from verification.

Overall, Task 1 established a functional STOMP baseline for subsequent robot adaptation and additional motion constraints.

Task 2 - Irfan

2.1 Robot Selection and Implementation

The Universal Robots UR5e, a 6-degree-of-freedom (6 DOF) serial manipulator, was selected to demonstrate the adaptability of the STOMP algorithm to a different robotic platform. The UR5e was chosen for its relatively simple 6 revolute joint configuration, its compatibility with MATLAB's Robotics System Toolbox, and suitability for subsequent Product of Exponential (PoE) implementation.

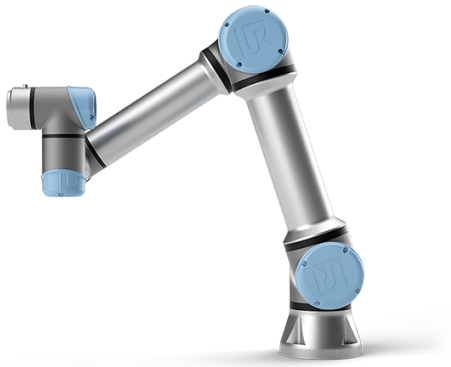


Figure 2.1: Universal UR5e manipulator robot selected

The Robotics System Toolbox was used to load the UR5e model. Initial and target end-effector poses were defined within the robot's reachable workspace, and inverse kinematics was used to determine the corresponding joint configurations.

The STOMP framework developed in Task 1 was adapted to accommodate the UR5e's six joints, updated robot geometry and collision representation. The trajectory was discretised into 20 waypoints, with 20 stochastic trajectories sampled per optimisation iteration.

2.2 Collision Modelling and Trajectory Optimisation

A collision model with 39 spheres was developed to approximate the UR5e's link geometry. The sphere centres were linked to their respective robot joints and transformed to world coordinates using forward kinematics. The STOMP objective integrates a velocity-weighted obstacle cost, heavily weighted at 1000, with a quadratic joint-space smoothness cost.

The initial linearly interpolated trajectory contained collisions at 7 of 20 waypoints. Following optimisation, STOMP converged after 10 iterations in 3.4 seconds, reducing the total trajectory cost from 178.709 to 43.346, a 75.7% reduction.

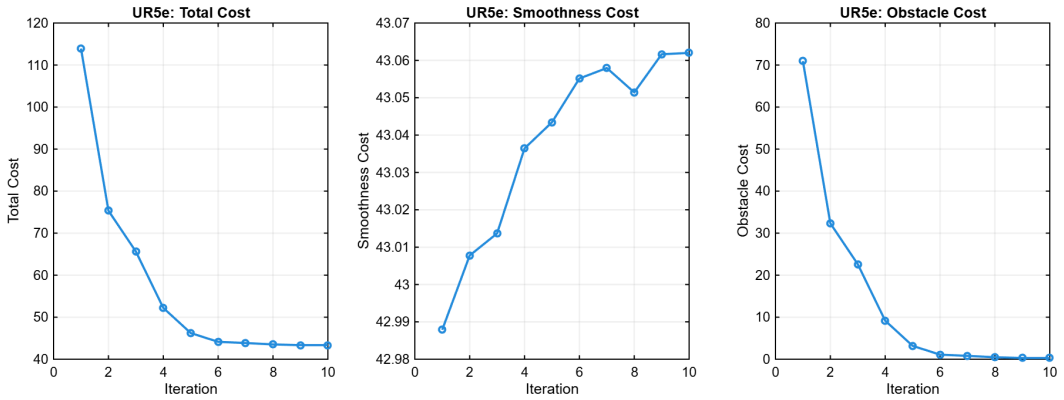


Figure 2.2: Convergence of Total, Smoothness, and Obstacle Cost During UR5e STOMP Trajectory Optimisation

As shown in Figure 2.2, the obstacle cost decreased significantly from 135.730 to 0.284 (99.8% reduction), while the smoothness cost remained relatively stable, increasing slightly from 42.979 to 43.062. This indicates that obstacle avoidance, rather than changes in trajectory smoothness, primarily drove the reduction in total cost.

The optimised trajectory achieved zero detected collisions across 191 densely interpolated configurations, with a minimum clearance of 38 mm according to the implemented collision model.

2.3 Stochastic Sampling Validation

To assess the STOMP algorithm's consistency, 10 trials were conducted with varying random seeds while keeping the initial and final configurations, obstacle geometry, and optimisation parameters constant. Each trial used 20 waypoints and 20 sampled trajectories per iteration,

achieving a 100% success rate in generating collision-free trajectories. The average optimisation runtime was 3.26 seconds with an average of 9.4 iterations, and the minimum obstacle clearance recorded was 26.4 mm. Most trials converged to similar trajectory costs around 43.3, though seed 30 resulted in a higher cost of 48.71 and the lowest clearance, highlighting the influence of stochastic sampling on the final optimised trajectory.

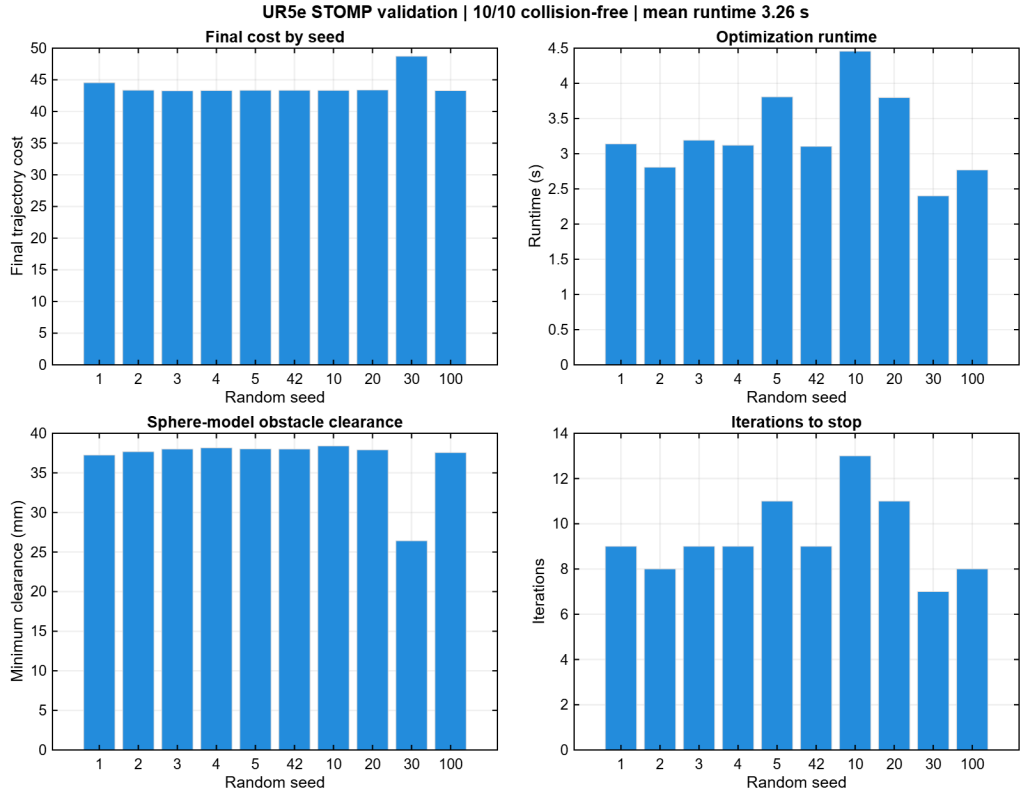


Figure 2.3. Validation of UR5e STOMP performance across ten random seeds, showing final trajectory cost, optimisation runtime, minimum obstacle clearance and convergence iterations.

Task 3 - Irfan

3.1 Product of Exponentials Formulation

The Product of Exponentials (PoE) formulation was implemented to replace MATLAB's built-in getTransform() function for forward kinematics calculations during STOMP trajectory optimisation.

For the 6-DOF UR5e manipulator, the end-effector transformation is expressed as:

Equation (1): Product of Exponentials

$$T(\theta) = e^{[S_1]\theta_1} e^{[S_2]\theta_2} e^{[S_3]\theta_3} e^{[S_4]\theta_4} e^{[S_5]\theta_5} e^{[S_6]\theta_6} M$$

Where $T(\theta)$ represents the end-effector pose, θ_i is the rotation angle of joint (i), (S_i) is its screw axis, and (M) is the home transformation matrix.

Each revolute joint is represented by a six-dimensional screw axis comprising angular and linear components:

Equation (2): Revolute Screw Axis

$$S_i = \begin{bmatrix} \omega_i \\ -\omega_i \times q_i \end{bmatrix}$$

where θ_i is the unit vector along the joint's rotation axis, q_i is a point on the axis expressed in the base frame, and v_i is the corresponding linear twist component.

The screw axis is converted into a 4×4 matrix representation for exponential transformation:

Equation (3): Twist Matrix

$$[S_i] = \begin{bmatrix} [\omega_i]_\times & v_i \\ 0_{1\times 3} & 0 \end{bmatrix}$$

The six screw axes and home transformations from the UR5e robot model at zero angle were precomputed and reused in STOMP optimisation for calculating the robot links' transformations.

3.2 UR5e Screw-Axis Derivation

The UR5e manipulator has ten rigid bodies linked by six revolute and four fixed joints. The revolute joints were identified using MATLAB's *showdetails()* function, with their axes and positions extracted at the zero-angle configuration.

For each joint, the local rotation axis and a point along the axis were transformed into the robot's base coordinate frame. The linear twist component was subsequently calculated using $v_i = -\omega_i \times q_i$. The resulting screw-axis components are summarised in Table X.

Joint	(θx)	(θy)	(θz)	(vx)	(vy)	(vz)
1	0	0	1	0	0	0
2	0	1	0	−0.1625	0	0
3	0	1	0	−0.1625	0	0.4250
4	0	1	0	−0.1625	0	0.8172
5	0	0	−1	−0.1333	0.8172	0
6	0	1	0	−0.0628	0	0.8172

Table 3.2. Screw-axis components of the six UR5e revolute joints.

The six screw axes were assembled into $S_{list} = [S_1\ S_2\ \dots\ S_6]$. The home transformation matrix (M) was obtained from the *tool0* end-effector pose at the zero-angle configuration:

$$M = \begin{bmatrix} -1 & 0 & 0 & 0.8172 \\ 0 & 0 & 1 & 0.2329 \\ 0 & 1 & 0 & 0.0628 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The screw axes and home transformations were initialised using ur5ePoEModel.m and reused for forward kinematics calculations. The PoE formulation for intermediate links included only the preceding joints and their corresponding home transformations, allowing for the calculation of link poses needed for collision modelling.

3.3 Numerical Validation

The PoE forward-kinematics accuracy was evaluated against MATLAB's *getTransform()* with 100 random UR5e configurations, comparing end-effector transformation matrices for errors in position and orientation.

Metric	**Mean Error**	**Maximum Error**
Position (m)	4.387×10^{-16}	1.195×10^{-15}
Rotation matrix (Frobenius norm)	6.133×10^{-16}	1.231×10^{-15}

Table 3.3. PoE forward-kinematics validation results across 100 joint configurations.

The errors were of the order of 10^{-15}, indicating numerical agreement between the PoE implementation and MATLAB's built-in forward kinematics.

Additional validation was performed on all six moving links and the *tool0* end-effector, with similarly negligible errors. These results confirm the accuracy of the PoE implementation for calculating the link transformations required during STOMP optimisation.

3.4 Computational Performance

The performance of the PoE implementation was tested against *getTransform()* function using 1,000 random joint configurations over five trials, calculating transformations for six moving links and the *tool0* end-effector.

Method	Mean Runtime (s)
MATLAB getTransform()	0.7835
PoE implementation	0.2472

Table 3.4. Forward-kinematics computational performance comparison.

The PoE implementation achieved a 3.17x speedup, corresponding to a 68.4% reduction in computation time compared to MATLAB's built-in forward kinematics.

This improvement can be attributed to the reuse of precomputed screw axes and home transformations, reducing repeated processing of the robot's kinematic structure. The benchmark excluded one-time model initialisation and therefore represents the computational performance of repeated forward-kinematics evaluations rather than the complete STOMP algorithm.

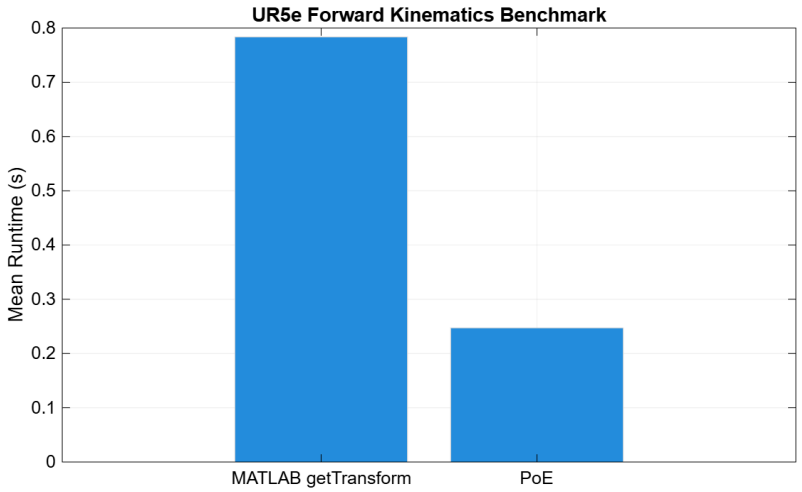


Figure 3.4. Comparison of mean forward-kinematics computation time between MATLAB's built-in getTransform() and the PoE implementation across five repeated trials.

3.5 Integration of PoE into STOMP

The PoE implementation was integrated into the STOMP framework by modifying MATLAB's updateJointsWorldPosition.m. Screw axes and home transformations were initialised in ur5ePoEModel.m, while ur5eFKinSpace.m computed link transformations during trajectory optimisation. This allowed the update of 39 collision spheres for obstacle-cost evaluation without built-in forward kinematics.

The PoE-STOMP planner converged in 1.5 seconds, reducing trajectory cost from 178.709 to 43.824 (75.5% reduction) and obstacle cost from 135.730 to 0.778, with a consistent smoothness cost. The optimised trajectory had zero collisions across 191 configurations and a minimum clearance of 38 mm. These results confirm successful PoE integration into STOMP while ensuring collision avoidance.

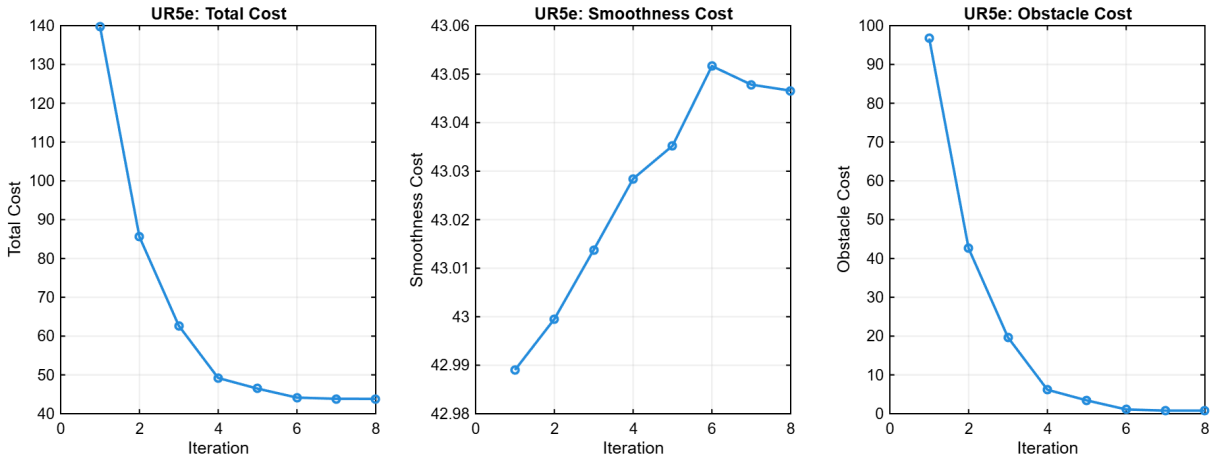


Figure 3.5. Convergence of total, obstacle and smoothness costs during UR5e STOMP trajectory optimisation using PoE forward kinematics.

Task 4 - Yash

4.1 Scenario design

The four scenarios each target a distinct failure mode of a local, sampling-based trajectory optimiser. Each scenario satisfies four requirements:

1. Reachable endpoints
2. Non - trivial initialisations
3. Endpoint clearance > 0.03m
4. Objects have realistic dimensions

Scenarios	Obstacles	Task	Why
Pillar	0.1 x 0.1 x 0.9 m cylinder, 0.55m in front of the base	Go from one side of the pillar to the other	Large deformation of the arm config far from initial position
Clutter	Divider, sphere, two cylinders, crate and an overhead bar	Go from one side to the other	Many obstacles, and tight clearances
Window	Wall 0.5m from the front with an opening of 0.36 x 0.36m	Reach through the window	Narrow passage, a small fraction of noisy samples lies inside the opening
Shelf	Two shelfs, with walls on side and back	Move from inside the lower shelf to the upper one	Long detour (retract, raise, re-insert) and far from straight line initialisation

Table 4.1 Scenarios

4.2 Initial results with the Task 1 stopping criterion

All four scenarios used the Task 1 STOMP, which stops when $\Delta Q < 0.1$ between iterations, or after 50 iterations.

- Total cost Q -
    - Obstacle cost: per waypoint, how far collision spheres penetrate, or come within 0.05 m of, obstacles. It is weighted by sphere speed and scaled by 1000, so it is zero with more than 0.05 m clearance.
    - Smoothness term: penalises abrupt path changes.

- Seeds - Each iteration samples 20 random perturbations of the trajectory, and the seed makes runs reproducible. Figures use seed 42. Success rates use three extra runs per scenario (seeds 1–3).
- Judging - The arm has 39 spheres which approximate its hitbox. Each result is rechecked against the real meshes with checkCollision at 191 configurations. A trajectory is collision-free only if none of the spheres collides.

Scenario	Colliding Waypoints in initial trajectory	Collision free runs	Mean Iterations
Pillar	8/20	0/3	27
Cluttered Tabletop	13/20	1/3	29
Window	11/20	0/3	20
Shelf	13/20	0/3	20

Table 4.2: Task 1 Stop Condition Collision Free Runs

Only 1 of 12 runs produced a collision-free trajectory. The runs stopped after only 20–29 iterations on average, well before the 50-iteration limit, so the limit was not what ended them.

The pillar reference run shows what happened (Figure 4.1):

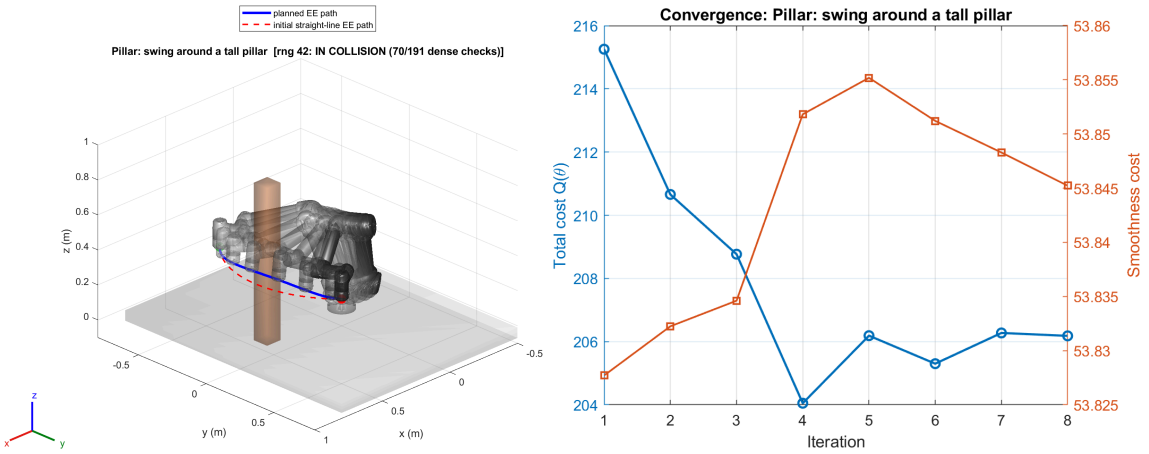


Figure 4.1: Task 1 Stopping Condition draws a path through the pillar and stops after 8 iterations

- The optimisation stopped after 8 iterations.
- At that point, the arm had moved less than 0.07 rad away from its initial trajectory and still passed straight through the pillar.

- The total cost was 206, of which about 150 was the obstacle cost. (See Appendix 4.1 for all Seed 42 results)

4.3 Why the Task 1 criterion fails, and the modified criterion

It fails because a change of cost of less than 0.1 can occur inside the obstacle itself. Because the update of STOMP points is random, Q fluctuates between iterations instead of falling steadily. In the pillar reference run, the first eight iterations gave Q = 215, 211, 209, 204, 206, 205, 206, 206: several units of noise with no real progress. The Task 1 test cannot distinguish a settled cost from two noisy values that happen to be close. In the pillar run, iterations 7 and 8 differed by 0.09, and the run stopped (Figure 4.1)

The new criterion (see Appendix 4.2 for code) no longer asks only "has the cost stopped changing?". It asks three questions:

1. Is the trajectory free of collisions and has the cost genuinely settled?
2. Has the optimiser stopped making progress altogether?
3. Has the iteration budget run out?

The new criterion uses three quantities:

- Q_k: total cost after iteration k.
- $d_{min,k}$: smallest sphere–obstacle gap over all waypoints, measured with the planner's sphere model and distance field (not the final mesh check). $d_{min,k} > 0$ means no sphere touches an obstacle, though the 0.05 m safety margin may still be violated.
- k^***:** the last iteration at which the best cost so far improved by more than 1%.

Optimisation stops at the first iteration k where any condition holds:

1. Converged: d_{min}, $k > 0$ and $|Q_k - Q_{(k-5)}| < 0.01 \cdot |Q_{(k-5)}|$ (from $k = 5$). The path is clear in the planners model, and Q has changed by under 1% over 5 iterations.
2. Stalled: $k - k^* \geq 30$. This ends runs stuck in a colliding local minimum where further iterations would not help
3. Iteration Limit: $k = 150$ (3 times old limit)
4. No change (same as Task 1)

4.4 Results with modified criterion

Scenario	Colliding Waypoints in initial trajectory	Collision Free Task 1 → modified	Mean Iterations, Task 1 → modified
Pillar	8/20	0/3 → 3/3	27 → 73
Cluttered Tabletop	13/20	1/3 → 3/3	29 → 46
Window	11/20	0/3 → 2/3	20 → 33
Shelf	13/20	0/3 → 3/3	20 →54

Table 4.3: Collision free results new criteria compared with task 1

In every scenario, the optimised trajectory differs substantially from the initial one, with maximum joint displacements of 0.58–0.74 rad, compared with less than 0.07 rad in the early-stopped pillar run:

- **Pillar:** the end-effector rises by about 0.3 m, and the arm passes around the pillar.
- **Clutter:** the gripper is lifted over the divider while staying below the overhead bar.
- **Window:** the wrist is routed through the opening on a curved path.
- **Shelf:** the arm retracts from the lower compartment before entering the upper one.

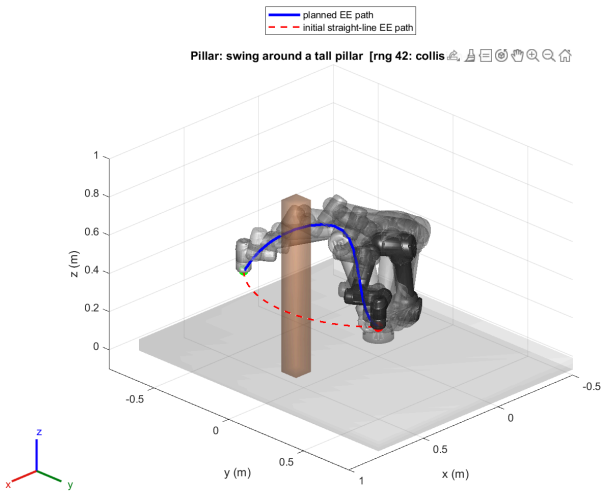


Figure 4.2. Task 4 difference between Task 1 stop criteria and modified

4.5 Limitations

1. STOMP only searches near the path it already has. Each iteration tries small, smooth random variations of the current trajectory (at most about 0.12 rad per waypoint) and moves towards the better ones. It can bend a path around an obstacle, but it cannot jump to a completely different route. So the further the solution is from the straight-line start, the longer it takes, and beyond some point it is never found:
    - Pillar: at 0.65 m from the base, 3/3 successes in 35 iterations on average; at 0.55 m, 3/3 but 73 iterations; at 0.42 m, where the arm must fold in tightly to pass, 0/3 (See Appendix for detailed test results)
    - Shelf: retracting from one compartment and re-entering the other needed 54 iterations, against 20 for the Task 1 runs that stopped early and failed.
2. Narrow passages are close calls. In the window scenario, the arm found the opening in every run, but one of three runs clipped the frame (2/3 collision-free).
3. The safety margin is not guaranteed. The 0.05 m margin only adds cost; it is not a hard limit. The planner trades it against a smoother path, so the final clearances were 0.021–0.035 m: collision-free, but closer than intended.
4. Longer planning time. Letting the optimiser finish needs 1.6–2.7× more iterations than the Task 1 criterion, which means 34–77 s per trajectory.

Task 5 - Yash

5.1 Goal and approach

In Task 5 the robot carries a cup that must not spill. Keeping the cup upright therefore means keeping the end-effector y-axis pointing straight up (along the world z-axis) for the whole motion.

The constraint is added as an extra cost that grows as the cup tilts, which STOMP then tries to keep small along with the obstacle cost. This is option 2 in the project brief. Because STOMP only needs to evaluate a cost, no other part of the planner has to change.

5.2 Cost

$$q_c(i) = w_c \, ||y_{ee}(\theta_i) - z_w||_1, \qquad S(i) = 1000 q_o(i) + q_c(i)$$

Where:

- y_ee is the end-effector's y-axis in world coordinates: the second column of its rotation matrix.
- z_w = $[0\ 0\ 1]^T$ is world "up".
- $\|\cdot\|_1$ adds up the absolute differences of the three components.
- w_c is the constraint weight: how strongly tilting is penalised compared with obstacles.
- q_o is the unchanged obstacle cost from Task 4.
- S(i) is the waypoint cost STOMP uses to rank its noisy samples.

The total cost Q and the update rule are otherwise unchanged.

Properties of this choice:

- Turning the cup about its own vertical axis costs nothing. Only tilting is penalised, so the planner keeps the freedom to rotate the cup if that helps avoid an obstacle.
- The penalty grows with tilt.
- It adds almost no computation. The end-effector pose comes from the same forward-kinematics call already made for the obstacle cost.
- Same start and goal poses

5.3 Scenarios and measurement

Task 5 reuses three of the Task 4 scenarios, so that the trajectory found with the constraint can be compared directly with the one found without it. A fourth scenario, a cup carried over a board, is used later to study failures (Section 5.6). All scenarios start and end with the cup upright.

Start and goal are chosen on the same arm configuration, with joint 6 near 0°. If one endpoint had joint 6 at +180° and the other at −180° (physically the same angle), the straight-line initial path would spin the wrist a full turn and turn the cup upside down.

How results are judged

- Tilt < 10 degrees: the angle between the end-effector y-axis and vertical, $\theta_{tilt} = \arccos(y_{ee} \cdot z_w)$. It is evaluated at the same 191 configurations, so tilt between waypoints is included.
- No Collisions
- Each scenario is tested constrained and unconstrained to see the difference in path and cost

5.4 Results: how the constraint changes the Task 4 paths (w_c = 10)

The main comparison uses w_c = 10. It lies inside the acceptable range, and compared with w_c = 3, it halves the clutter tilt (7.3° → 3.3°), turns the clutter result from 2/3 to 3/3 collision-free, and costs only about 10% more iterations (89 → 98 in clutter, 69 → 73 in the pillar). Above w_c = 10, the tilt hardly improves (0.6–1.5°), while the pillar needs almost twice the iterations (73 → 122). (For results for each weight w_c, see Appendix 5.1)

Scenario	Collision Free, w_c = 0	Median Max Tilt, w_c = 0	Mean Iterations	Collision - free w_c = 10	Median Max Tilt, w_c = 10	Mean Iterations
Window	2/3	25.3°	37	2/3	1.5°	43
Pillar	3/3	15.5°	50	3/3	0.6°	70
Cluttered	2/3	37.9°	28	3/3	3.3°	150

Table 5.1: Comparison of how weight w_c affects upright runs on seeds 1 to 3

Across all three scenarios, the constraint reduces the cup tilt from 15–38° to 0.6–3.3°, and no collision-free runs are lost; clutter even gains one. (See Appendix 5.2 for images of comparisons for the above)

- Pillar (A5.2.1). Without the constraint, the planner lifts the cup to about 0.6 m and rolls the wrist to swing it around the pillar, tipping the cup by up to 44°. With the constraint, the path stays lower (about 0.4 m), and the wrist stays level, so the arm passes the pillar with the cup upright.
- Clutter (A5.2.2). Both paths arc over the divider and under the overhead bar. Without the constraint, the cup tips by up to 37°, and the path passes within 1 mm of an obstacle. With the constraint, the cup stays within 2.3°, and the clearance improves to 0.042 m. This was the hardest optimisation: it ran to the 150-iteration limit, where it was collision-free but its cost had not fully settled.
- Window (A5.2.3). This is the Task 4 path itself next to its constrained version. The tilt drops from 14.6° to 2.4°, but the constrained path passes much closer to the window frame (0.004 m instead of 0.035 m). Holding the cup level leaves less freedom to keep the arm away from the frame.

5.5 Failure created by the constraint

The board scenario was built to find the opposite situation, where the unconstrained planner succeeds, and the constraint makes it fail. The board height was screened with the constraint on: The high board (top at 0.33 m) is the failure case. Its three versions are compared below:

High Board (Seed 1 to 3)	Collision - Free	Median Max Tilt	Mean Iterations
No constraint	2/3	43.2°	36 (reference run)
W_c = 10, straight line start	0/3	2.5°	30
W_c = 1, via point start	3/3	1.8°	31

Without the constraint, the planner lifts the cup in a high arc over the board, tilting it by up to 48° on the way (Figure 5.1, left). With the constraint, the planner instead tries to slide the cup *under* the board (Figure 5.2, right). The reason is in the cost: staying low and level costs nothing in the constraint term, while lifting the cup over the board means passing through

arm configurations that tip it, which the constraint penalises. The low route is a local minimum. Every variation STOMP tries from there is worse, so it stops after only about 30 iterations with the arm still in collision and the cup perfectly upright. The constraint has closed off the escape route the unconstrained planner used.

The fix is a better starting path. STOMP was started from a path through one via-point above the centre of the board (0.62, 0, 0.48 m), cup upright, built as two straight joint-space segments. From this start, all 3 runs were collision-free, with the cup within 1.8° of upright (Figure 5.1, bottom). The task is therefore feasible. What fails is the straight-line initialisation combined with the constraint. This is the same local-minimum weakness noted in Section 4.4, made worse by the constraint, and it shows why STOMP is usually paired with a coarse global planner that supplies the initial path.

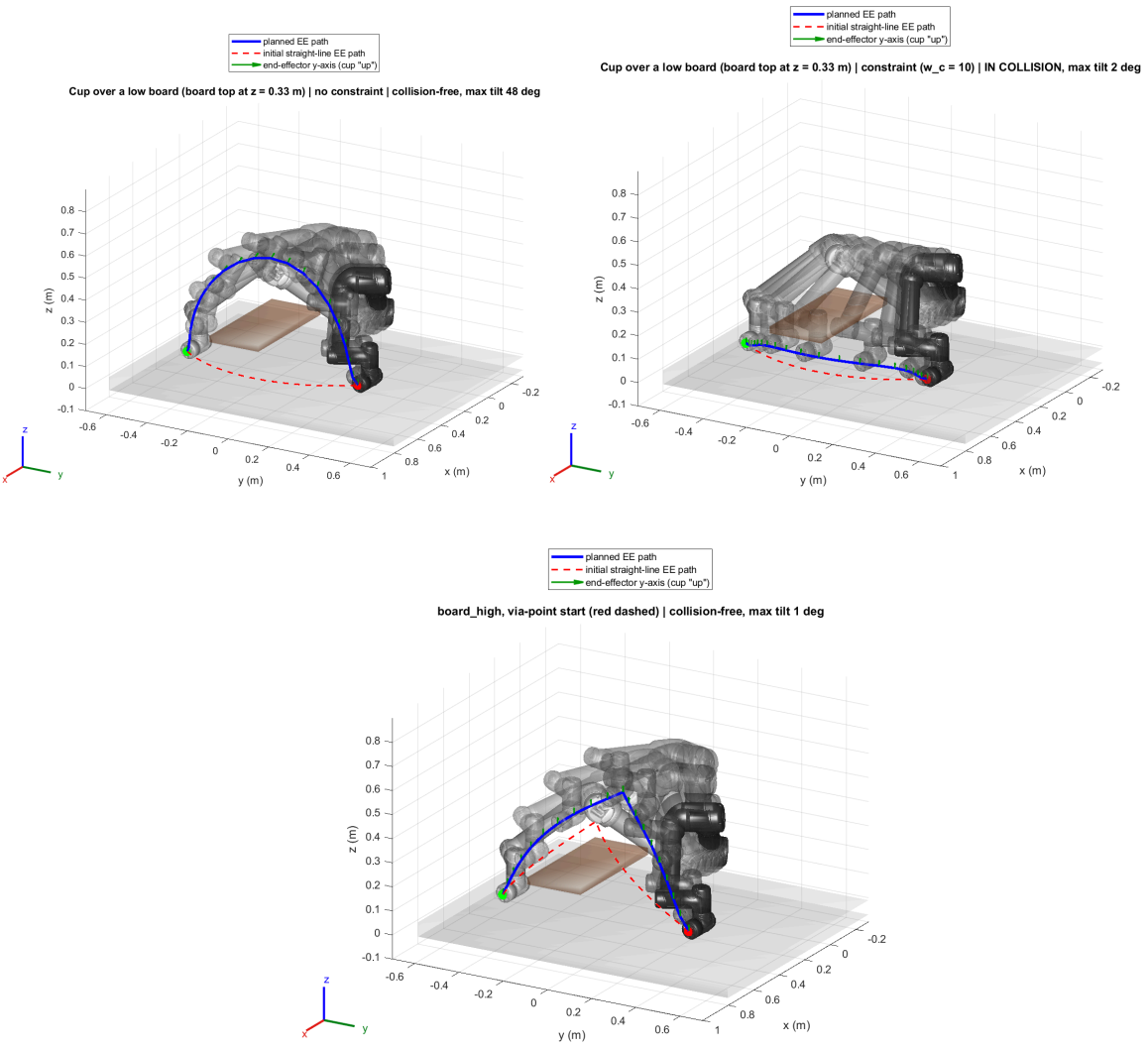


Fig 5.1

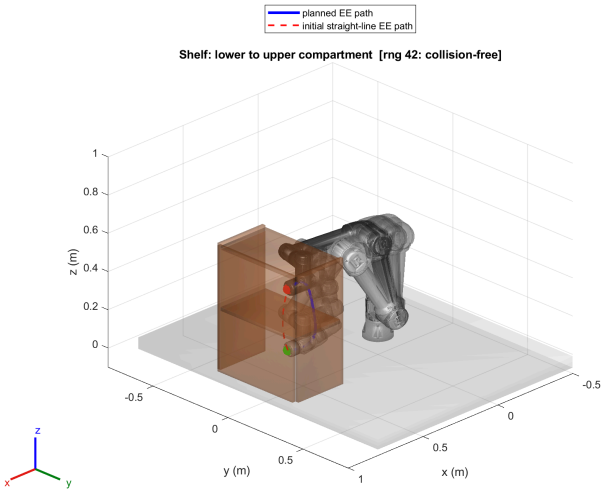
5.6 Limitations

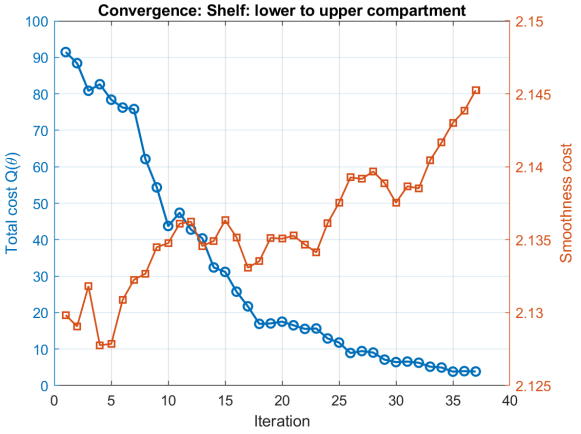
- The constraint is a penalty, not a guarantee. The cup still tilts slightly (median up to 3.3° in clutter), and the amount depends on the chosen w_c. A task with a strict limit, such as a full cup that must never exceed 5°, cannot be certified this way. That would need a hard constraint enforced in the update itself.
- The constraint can trap the planner. Over the high board, the unconstrained planner escaped by tilting the cup over the board. With the constraint, that route became expensive, so the planner settled under the board in collision (0/3). The constraint therefore makes STOMP more dependent on a good initial path. A via-point start fixed this case (3/3), but in practice a global planner would be needed to supply that path automatically.
- Keeping the cup level reduces clearance. Without the freedom to tilt, the arm has fewer ways to move away from obstacles. In the window, clearance dropped from 0.035 m to 0.004 m. A margin that small leaves no room for model or tracking errors on a real robot.

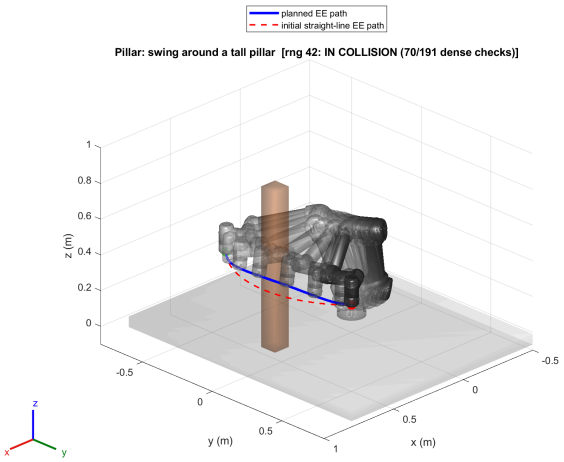
Appendix

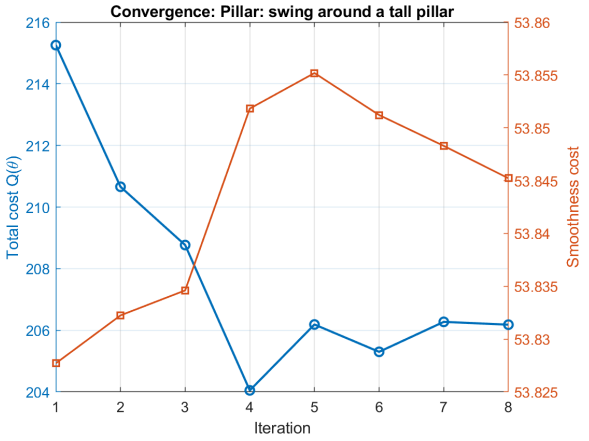
A4.1 Seed 42 Results Comparison

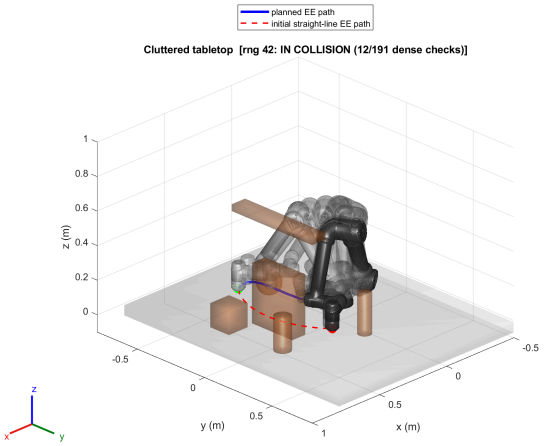
Task 1 Stopping Cost Runs (seed 42)

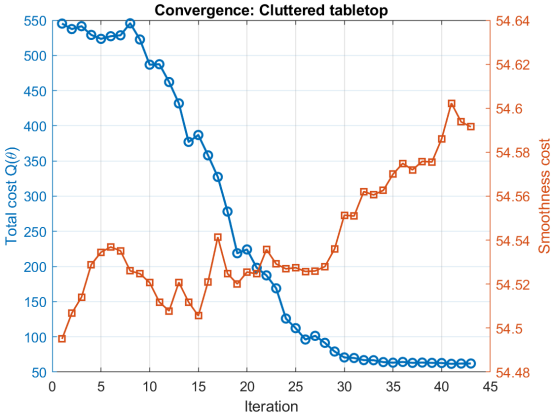


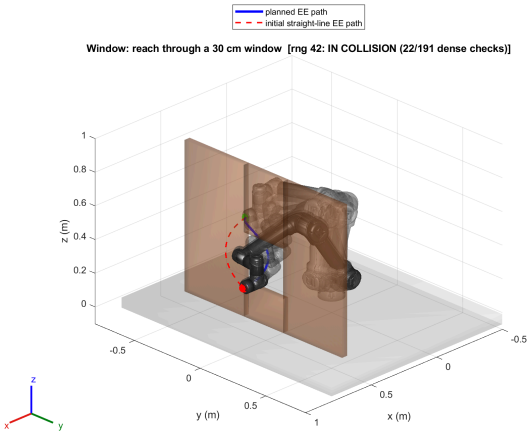


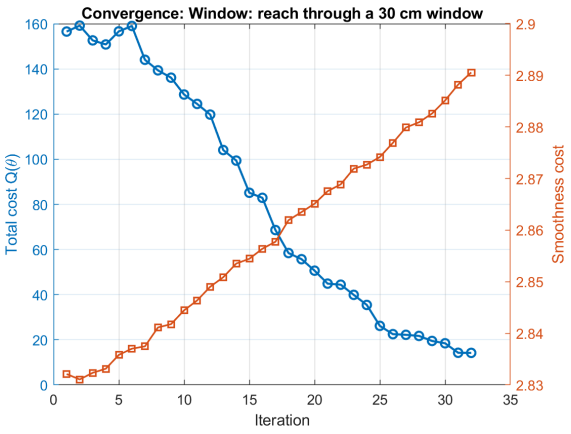




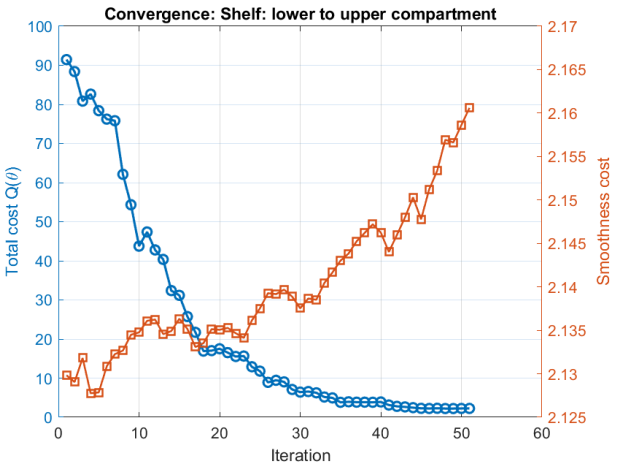


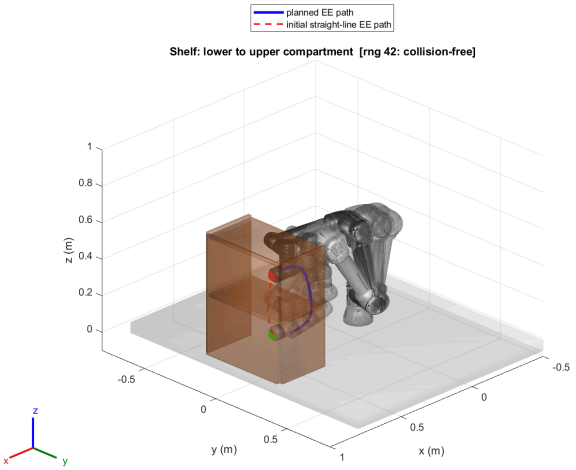


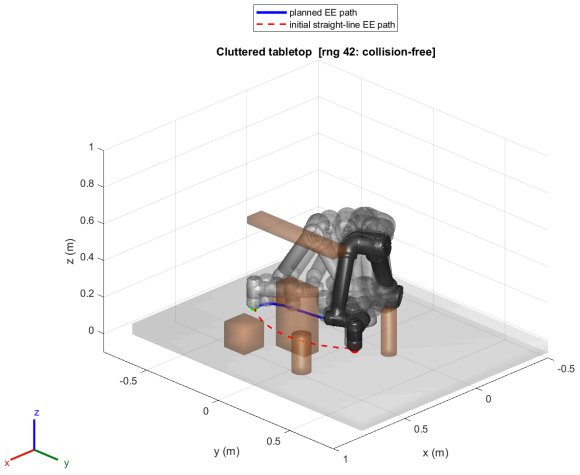


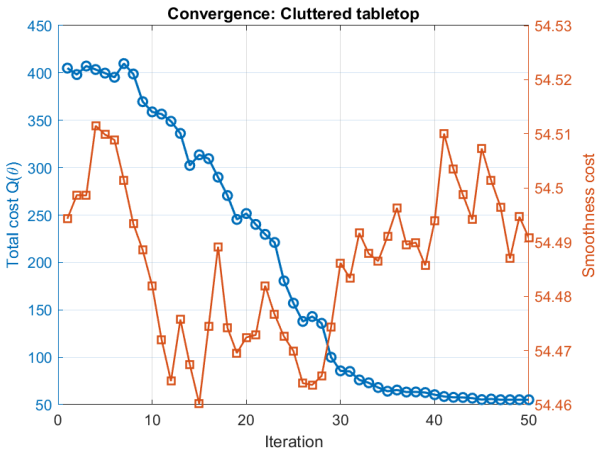


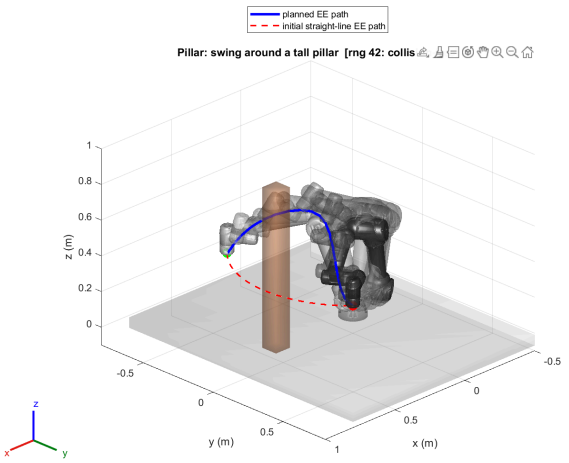
New Stopping Criteria Results (Seed 42)

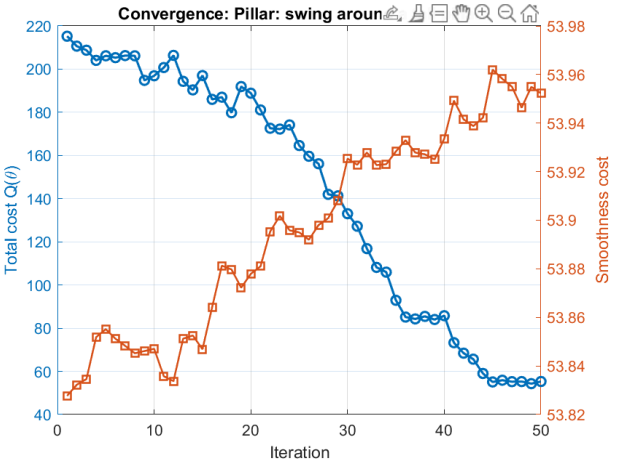


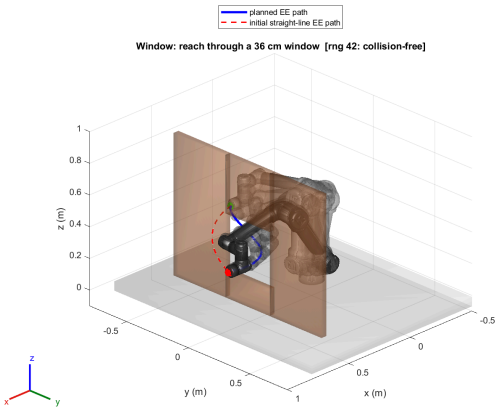


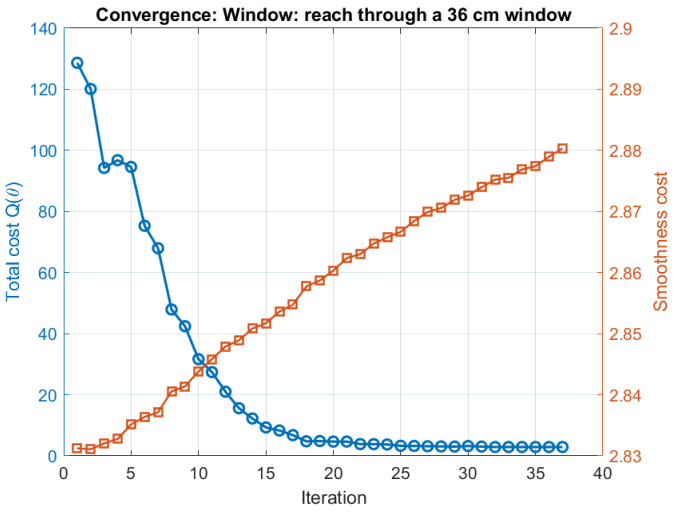





planned EE path
initial straight-line EE path
Pillar: swing around a tall pillar [rng 42: collis
z (m)
y (m)
x (m)


Convergence: Pillar: swing aroun
Total cost Q(θ)
Smoothness cost
Iteration


planned EE path
initial straight-line EE path
Window: reach through a 36 cm window [rng 42: collision-free]
z (m)
y (m)
x (m)


Convergence: Window: reach through a 36 cm window
Total cost Q(θ)
Smoothness cost
Iteration

A4.2 Code for stopping rule

```
    % ---- stopping rule ----
    if norm(dtheta_smoothed, 'fro') < 1e-10
        stopReason = 'update ~ 0';
    elseif strcmpi(stopRule, 'original')
        if abs(Qtheta - QthetaOld) <= convergenceThreshold
            stopReason = 'converged (|dQ| < threshold)';
        elseif iter >= maxIterations
            stopReason = 'maximum iterations';
        end
    else
        if Qtheta < (1 - relTol) * bestQ
            bestQ = Qtheta; bestIter = iter;
        end
        Qhist = [Qtheta_initial, Q_time];
        settled = iter >= patience && ...
            abs(Qhist(end) - Qhist(end - patience)) < relTol * abs(Qhist(end - patience));
        if settled && stompSphereClearance(robot, theta, voxel_world, model) > 0
            stopReason = 'converged (sphere model collision-free, cost settled)';
        elseif iter - bestIter >= stallIters
            stopReason = sprintf('stalled (no %g%% improvement in %d iterations)', 100 * relTol, stallIters);
        elseif iter >= maxIterations
            stopReason = 'maximum iterations';
        end
    end
end
```

A5.1 Choosing the constraint weight

The weight w_c sets the balance between keeping the cup upright and avoiding obstacles. Too small and the cup still tips; too large and the planner may neglect obstacles or take much longer. Rather than tuning by hand, a fixed rule was applied. w_c was tested at 0, 3, 10 and 30 on the two scenarios where the unconstrained cup tilts most.

A weight is acceptable if, in both scenarios:

1. Tilt <= **10°**
2. Runs with constraints should have more collision free runs than without constraints

w_c	Successful trials	Median Max Tilt	Mean Iterations	Acceptables
0	2/3	37.9	67	No
3	2/3	7.3	89	Yes
10	3/3	3.3	98	Yes
30	3/3	1.5	96	Yes

Every weight from 3 to 30 is acceptable, and **w_c = 3** is the smallest that is acceptable.

A5.2 Comparison Pictures of no constraint vs w_c = 10

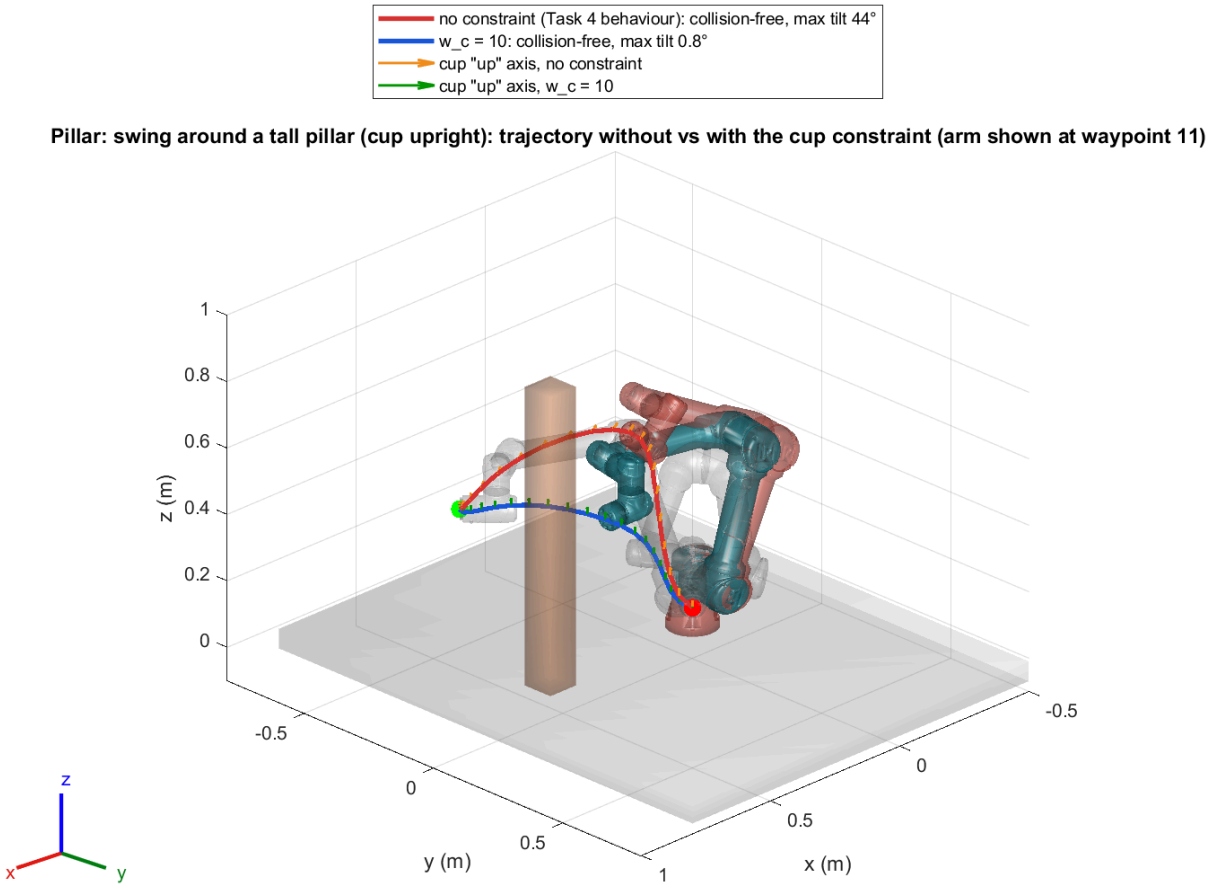


Fig A5.2.1: Pillar Seed 42, no constraint vs constraint path difference

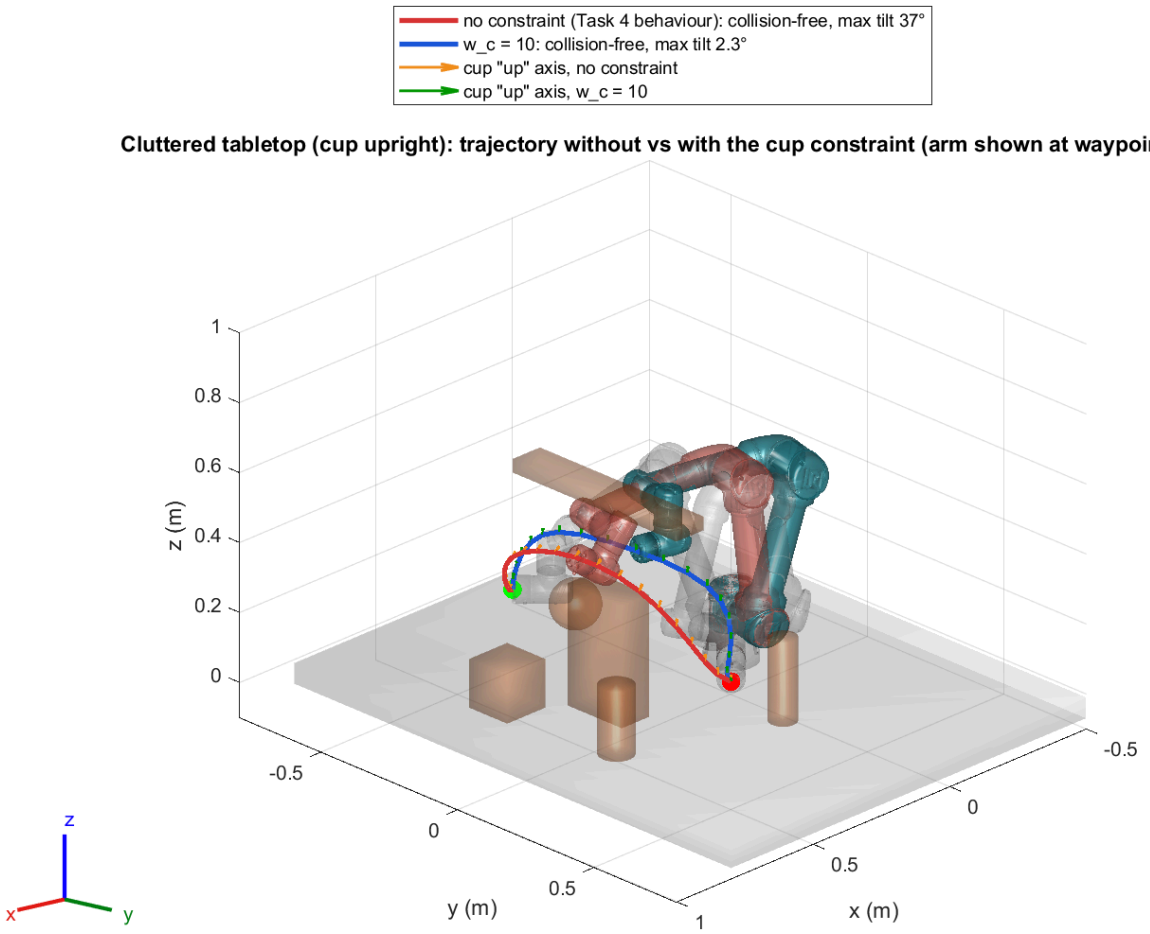


Fig A5.2.2: Clutter Seed 42, no constraint vs constraint path difference

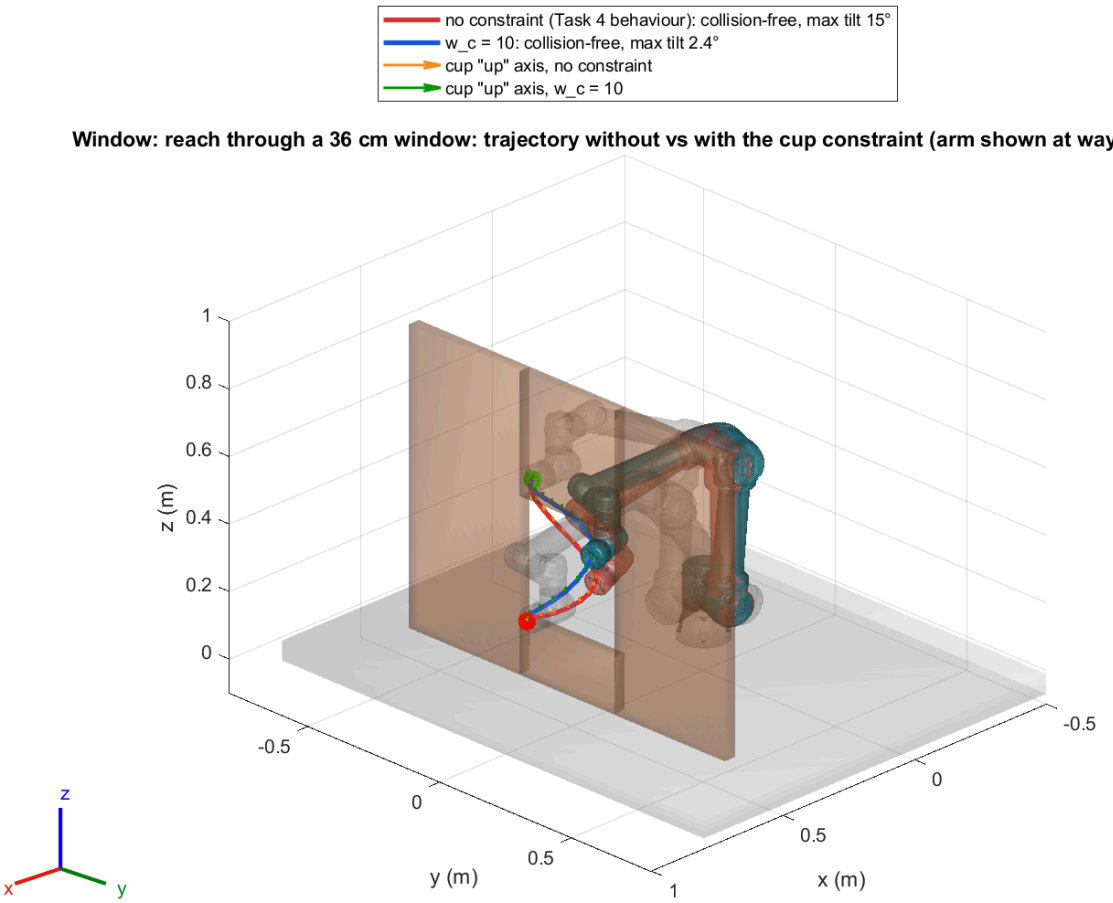
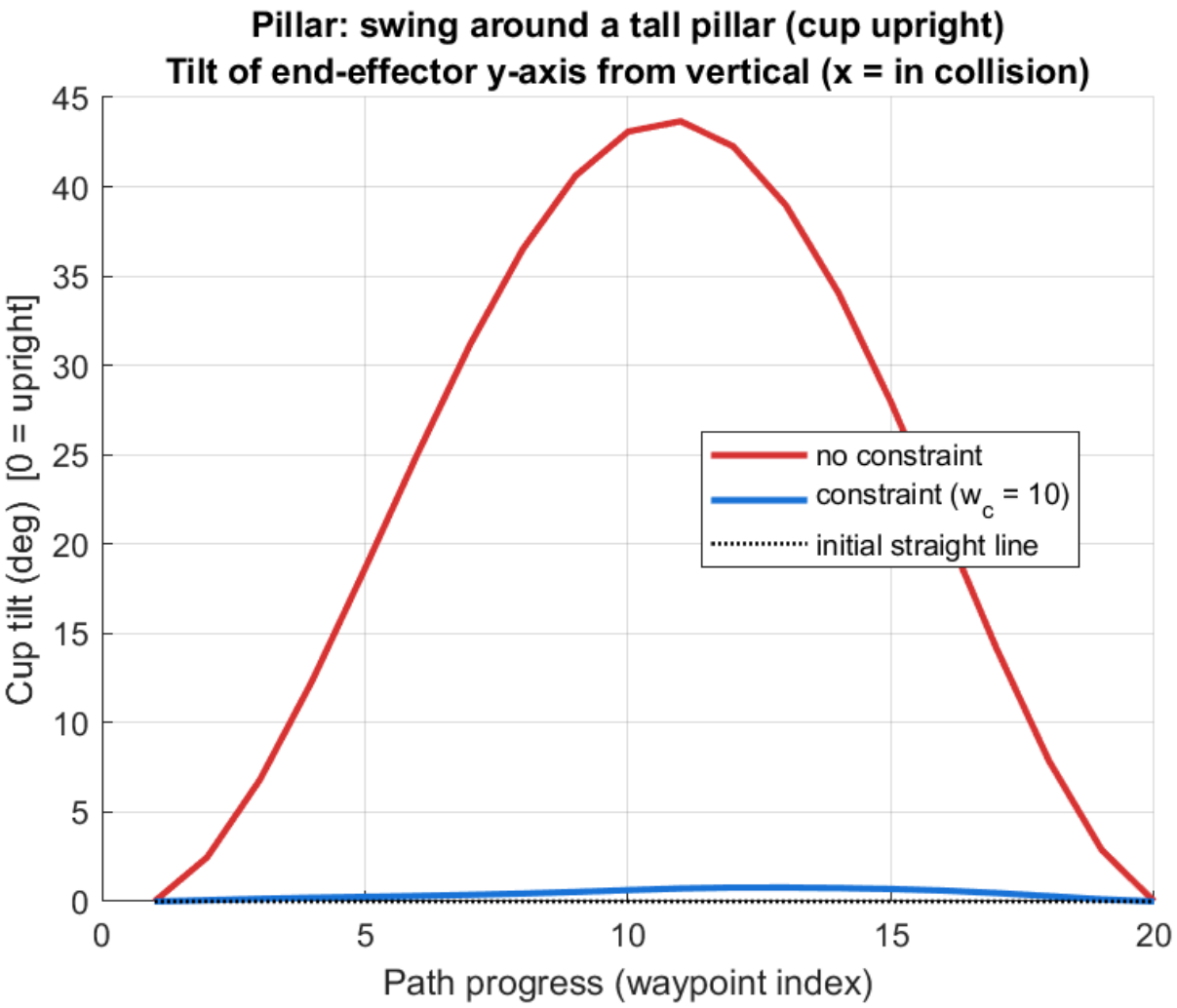
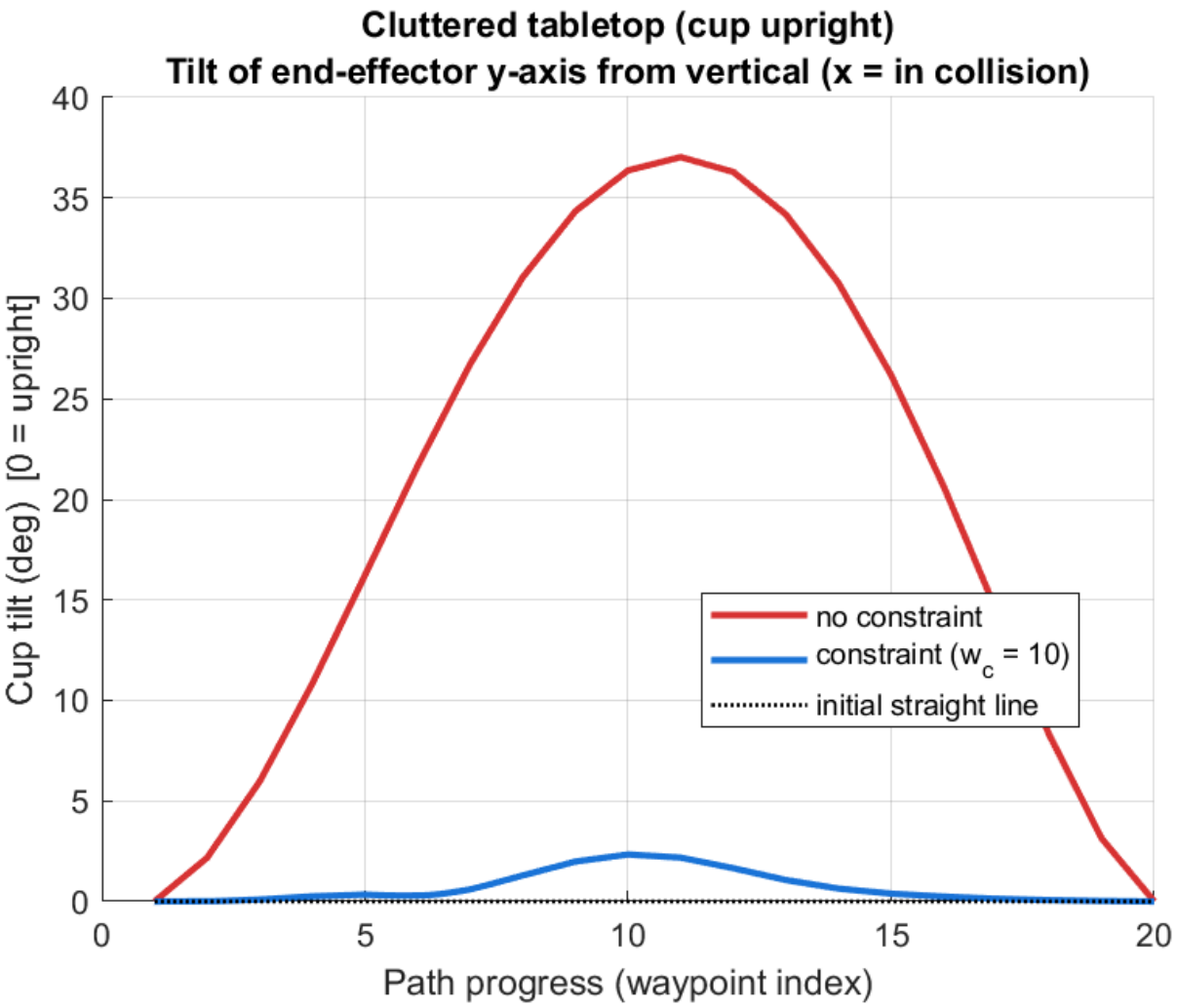
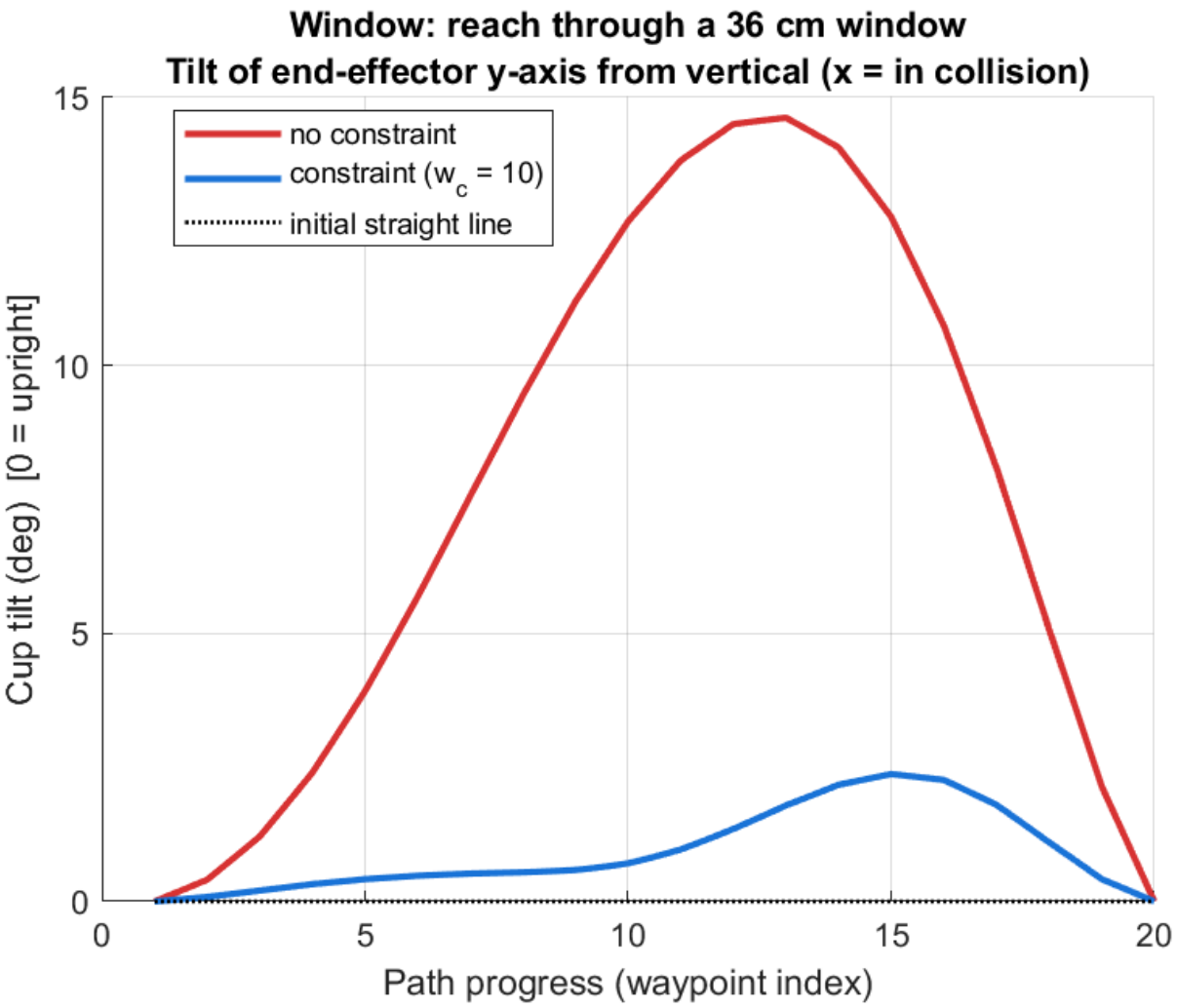


Fig A5.2.3: Window Seed 42, no constraint vs constraint path difference


Pillar: swing around a tall pillar (cup upright)
Tilt of end-effector y-axis from vertical (x = in collision)
Cup tilt (deg) [0 = upright]
Path progress (waypoint index)
no constraint
constraint (w_c = 10)
initial straight line
0
5
10
15
20
25
30
35
40
45


Cluttered tabletop (cup upright)
Tilt of end-effector y-axis from vertical (x = in collision)
Cup tilt (deg) [0 = upright]
Path progress (waypoint index)
no constraint
constraint (w_c = 10)
initial straight line
0
5
10
15
20
25
30
35
40


Window: reach through a 36 cm window
Tilt of end-effector y-axis from vertical (x = in collision)
no constraint
constraint (w_c = 10)
initial straight line
Cup tilt (deg) [0 = upright]
Path progress (waypoint index)
0
5
10
15
20